

## Initial Project Overview

The Restaurant Booking System will be an online website which will simplify the process of reserving tables. To book a table, customers simply need to enter the number of guests in their party after which the website will display available time slots to choose from. After picking their preferred time they will confirm the reservation and receive a confirmation email or message. This website will improve the booking experience for customers.

In contrast, as a restaurant owner / admin you will have an extra set of functions & permissions on your account. You will be given the ability to see the level of stock you have, by this I mean that for example burgers that you make in house you could see the stock of mince you have before a day & at the end of the night. This is useful as owners are expected to use our website quite a lot so having this feature somewhere relevant is a quality of life that will be useful. Also, when stock becomes low the owner will receive an email as well as a notification on the web app letting them know stock is getting low. Another feature that owners will have is the ability to see which tables are booked in real time, they will also be able to see how many parties (groups) will be attending as well as how many people are expected to be coming. This real time information will be a good implementation, allowing owners to be efficient in how many staff members they put on shift that day ultimately allowing them to be smart with their staff's time as well as his money.

Furthermore, another ability an owner will have is being able to see which table have allergies and or vegan, vegetarian etc... This will allow prior preparation for people with specific needs to give the best experience possible.

## Features

- **Home Page (Easy)** : Why it's easy: It's mostly static content, with basic HTML for structure and CSS for styling. There's minimal interactivity, so you don't need much logic. Tasks: Create a welcoming home page with basic information about the restaurant and links to other pages.
- The non static content could come in the form of a hotbar (wherever that may go), will have various icons which take you to various parts of the web app & the part that will make this dynamic is that if you are an owner you can expect more icons as you have access to specialist tools through the web app.
- **Menu Display (Easy to Moderate)**: Why it's fairly easy: Displaying a list of menu items (e.g., food name, description, price) using HTML and CSS is simple. If you're fetching data from the back-end API, that adds a bit of complexity, but it's still straightforward. Tasks: Fetch menu items from a back-end API and display them. Use CSS to make the menu look attractive.
- Menu data will be stored in a table within our MySQL database, models will be made to send the data to a controller corresponding to a specific menu (or menu part). 2 ways of retrieving images for menu, either store images in same folder as project & link them into the web page or store them on MySQL database & call them (get) as normal.

- **Login & Registration (Moderate).** Why it's moderate: User authentication involves forms and user data validation, which are more complex than displaying static content. You'll need to set up user sessions (to keep users logged in) and handle login/logout. Tasks: Create forms for user login/registration, store user details in the database, and handle authentication using back-end logic (like password hashing and user sessions).
- Keep in mind ethics, do not store people's data that we do not need for example we really only need age, name, email and or phone number
- Treat dummy data as if it is real data, to keep integrity when programming this web app
- **Reservation System (Moderate)** - Why it's moderate: While the form itself is simple, handling reservations involves interacting with the database to check availability and store reservation details. Tasks: Create a reservation form, validate user inputs, and send the reservation data to the back-end API. Store reservation details in a database and send confirmation.
- Information expected to be updated in real time, for example if someone is trying to book a table either make it so no more than one person can select the same table at a time or make it so if one person books it first then the other user cannot book that table for that time
- **User Roles (Moderate to Advanced):** Why it's moderate to advanced: Handling different access levels (e.g., customer vs admin vs staff) requires adding role-based access control to certain features. This adds complexity to user authentication and authorization logic. Tasks: Assign different permissions to users based on their role and restrict certain pages to only admins or staff.
- **Inventory Management (Moderate)**
  - Tasks:
  - Create an inventory management page for staff/admin.
  - Allow staff/admin to:
    - **Add new inventory items:** Fields for item name, quantity, and unit (e.g., kg, litres).
    - **Edit existing inventory items:** Ability to modify item details.
    - **Delete inventory items:** Functionality to remove items from the inventory.
  - Display a list of current inventory items with stock levels.
  - Implement low stock alerts:
    - Notify staff when stock falls below a defined threshold (e.g., send an email and or display a warning on the dashboard (via popup perhaps)).
- **Billing and Transactions (Moderate to Advanced)**

**Why it's moderate to advanced:** Generating bills and handling transactions require careful management of financial data and integration with payment gateways.

**Tasks:**

- Create a billing page that displays:
  - Order details (items purchased, prices).
  - Total amount due.
- Implement functionality to generate and display bills after an order is completed.
- Integrate payment processing:
  - Use a payment gateway API (e.g., Stripe, PayPal) to handle online payments.
  - Ensure secure data transmission and storage of sensitive information.
- Provide users with a transaction history page:
  - Allow users to view their past payments and bills.

- **Admin Panel (Moderate to Advanced)**

**Why it's moderate to advanced:** The admin panel requires a range of features to manage the application effectively and monitor various metrics.

**Tasks:**

- Create a dedicated admin panel accessible only to admin users.
- Dashboard functionalities:
  - Display key metrics (total orders, revenue, current reservations).
  - Provide charts or graphs for visual representation of data.
- User management features:
  - View and manage all user accounts (customers and staff).
  - Edit user roles and permissions.
  - Deactivate or delete user accounts if necessary.
- Reporting features:
  - Generate reports for sales, inventory levels, and customer feedback.
  - Export reports to formats like CSV or PDF for external use.

- **Customer Services (Moderate)**

**Why it's moderate:** Providing customer services involves creating user-friendly interfaces and systems to assist users, which can require some logic for form handling and feedback collection.

**Tasks:**

- Create a "Contact Us" page with:
  - A contact form for customer inquiries (fields for name, email, message).
  - A display of contact information (phone number, email, physical address).
- Develop an FAQ section:
  - Populate it with common questions and answers regarding the restaurant, orders, and reservations.
- Implement a feedback system:
  - Allow users to submit feedback or reviews on their dining experience.
  - Display feedback on the website for potential customers to view.

- **User Interface Features (Easy to Moderate)**

**Why it's easy to moderate:** Enhancing the user interface will improve usability and aesthetics, requiring some level of design and interaction.

**Tasks:**

- Implement responsive design:
  - Use CSS media queries to ensure the website works well on various devices (desktop, tablet, mobile).
- Improve user experience with:
  - Smooth transitions and animations for menu items and forms.
  - Clear navigation and layout for easy access to different sections of the website.
- Add interactive elements:
  - Use JavaScript for dynamic updates, such as live search functionality in the menu or real-time reservation availability checking.

- **Security Features (Moderate to Advanced)**

**Why it's moderate to advanced:** Implementing security measures is critical for protecting user data and ensuring secure transactions, which involves understanding various security protocols.

**Tasks:**

- Implement data encryption:
  - Use HTTPS for secure data transmission.
  - Hash passwords before storing them in the database.
- Set up session management:
  - Ensure users are logged out after a certain period of inactivity.
  - Protect against session hijacking and CSRF attacks.
- Regularly update dependencies:
- Monitor and patch any vulnerabilities in libraries or frameworks used in the project.

- **Version Control (Easy)**

**Why it's easy:** Setting up version control is straightforward and essential for collaborative development.

**Tasks:**

- Set up a Git repository for the project:
  - Initialise the repository and commit initial code.
- Create branches for different features or bug fixes:
  - Encourage team members to use feature branches and pull requests.
- Maintain clear commit messages and documentation:
  - Ensure all changes are well-documented for future reference.

Data privacy, GDPR & generic guidelines:

- Hashed passwords kept on server (only person who can see plaintext password is end user)
- Only collect data that is relevant to our project
- Remember that under collection of information can be better than collecting too much information